Лабораторная работа №5
«Программирование шины USB»

1. Цель работы

Изучить архитектуру шины USB, её топологию, версии и роли хост-контроллеров.

2. Задачи

1. Изучить архитектуру шины USB, её топологию, версии (1.1–USB4) и роли хост-контроллеров (OHCI/UHCI, EHCI, xHCI).
2. Освоить форматы дескрипторов USB-устройств (Device, Configuration, Interface, Endpoint) и механизм перечисления (enumeration).
3. Изучить отличия составных (*composite*) и композитных (*compound*) устройств.
4. Получить практические навыки работы с низкоуровневыми API: libusb, libudev (Linux), SetupAPI / CfgMgr32 (Windows).
5. Научиться получать идентификационную информацию (VID/PID, строковые дескрипторы, классы), отслеживать события hotplug и выполнять программное безопасное извлечение устройств.
6. Освоить тестирование приложений в эмуляторе QEMU с пробросом реальных USB-устройств.

3. Теоретические сведения

3.1. Архитектура шины USB

Шина USB построена по топологии **многоуровневой звезды** (*tiered-star*). Корневые хабы (*root hubs*) интегрированы в контроллеры хоста, к которым через внешние хабы подключаются конечные устройства.

Параметр	USB 1.1	USB 2.0	USB 3.x	USB4
Скорость	12 Мбит/с	480 Мбит/с	5–20 Гбит/с	до 40 Гбит/с
Хост-контроллер	OHCI/UHCI	EHCI	xHCI	xHCI + TB
Сигнальные линии	2 (D+, D−)	2 (D+, D−)	6 (+ SSTX, SSRX)	USB4/TB

Ключевые понятия:

- **Endpoint** — логическая точка передачи данных на устройстве (IN/OUT).
- **Pipe** — логический канал между хостом и endpoint.
- **Enumeration** — процесс опознания устройства хостом после подключения: сброс порта, назначение адреса (1–127), чтение дескрипторов, выбор конфигурации, загрузка драйвера.

3.2. Дескрипторы USB-устройств

Каждое USB-устройство предоставляет иерархию дескрипторов, запрашиваемых хостом при enumeration:

Интерфейсы и инструментальные средства вычислительных машин

Тип	Размер	Ключевые поля
Device	18 байт	idVendor, idProduct, bcdDevice, bDeviceClass, iManufacturer, iProduct, iSerialNumber, bNumConfigurations
Configuration	9+ байт	wTotalLength, bNumInterfaces, bConfigurationValue
Interface	9 байт	bInterfaceClass, bInterfaceSubClass, bInterfaceProtocol, bNumEndpoints
Endpoint	7 байт	bEndpointAddress, bmAttributes, wMaxPacketSize, bInterval
String	2+ байт	Язык, Unicode-строка

Основные классы устройств (bInterfaceClass):

Класс	Значение	Примеры
Audio	0x01	Наушники, микрофоны
CDC	0x02	Модемы, последовательные порты
HID	0x03	Мышь, клавиатура, геймпад
Mass Storage	0x08	USB-флешки, внешние HDD
Hub	0x09	USB-хабы
Video	0x0E	Веб-камеры (UVC)
Smart Card	0x0B	Ридеры смарт-карт

3.3. Составные устройства (Composite vs Compound)

Характеристика	Composite Device	Compound Device
USB-адрес	Один	Несколько (через внутренний хаб)
Дескрипторы	Одна Configuration с несколькими Interface	Несколько независимых Device Descriptor
Примеры	Клавиатура + тачпад, гарнитура	Монитор с USB-хабом, docking station
Обработка в ОС	Один драйвер или несколько (IAD)	Несколько независимых драйверов

3.4. Механизмы низкоуровнего доступа в ОС

Linux

Интерфейс	Назначение
sysfs (/sys/bus/usb/devices/)	Иерархия файлов с VID, PID, manufacturer, product, serial, конфигурациями
libusb 1.0	Пользовательский доступ к USB: чтение дескрипторов, control transfers
libudev	Мониторинг hotplug-событий в реальном времени
udisks2 (D-Bus)	Безопасное извлечение, монтирование/размонтирование томов

Ключевые функции **libusb 1.0**:

Интерфейсы и устройства вычислительных машин

- libusb_init / libusb_exit — инициализация контекста.
- libusb_get_device_list / libusb_free_device_list — получение списка устройств.
- libusb_get_device_descriptor — чтение дескриптора устройства (18 байт) без открытия.
- libusb_open / libusb_close — открытие устройства для I/O.
- libusb_get_string_descriptor_ascii — получение строковых дескрипторов.
- libusb_get_config_descriptor / libusb_free_config_descriptor — чтение конфигурации с интерфейсами.

Ключевые функции **libudev**:
- udev_new / udev_unref — контекст.
- udev_monitor_new_from_netlink — создание монитора событий.
- udev_monitor_receive_device — получение события (add/remove/change).
- udev_device_get_sysattr_value — чтение атрибутов sysfs.

Windows

API / Механизм	Назначение
SetupAPI (setupapi.h)	Перечисление устройств PnP, получение свойств реестра
CfgMgr32 (cfgmgr32.h)	Управление конфигурацией, безопасное извлечение (CM_Request_Device_Eject)
WMI (Win32_USBHub, Win32_PnPEntity)	Получение свойств устройств через COM-интерфейс
WM_DEVICECHANGE	Оконное сообщение о подключении/отключении устройства

Ключевые функции **SetupAPI**:
- SetupDiGetClassDevs — получение набора устройств по GUID.
- SetupDiEnumDeviceInterfaces — перечисление интерфейсов.
- SetupDiGetDeviceInterfaceDetail — получение DevicePath.
- SetupDiGetDeviceRegistryProperty — чтение свойств реестра.
- SetupDiDestroyDeviceInfoList — освобождение набора.

Ключевые функции **CfgMgr32**:
- CM_Locate_DevNode — поиск узла устройства по Instance ID.
- CM_Request_Device_Eject — запрос на безопасное извлечение (возвращает CR_REMOVE_VETOED при отказе).

3.5. Безопасное извлечение

Безопасное извлечение гарантирует сброс кэшей файловой системы, завершение транзакций записи и отключение питания с порта.

Причины отказа: 1. Открытые файловые дескрипторы на томах устройства. 2. Активные процессы, использующие директории на устройстве. 3. Недостаточные права доступа (требуются root / администратор). 4. Отсутствие драйвера, корректно поддерживающего IRP_MN_EJECT (Windows). 5. Один из интерфейсов составного устройства занят системой.

4. Варианты заданий

Интерфейсы устройства вычислительных машин

Каждый студент выполняет **строго один** вариант. Реализация выполняется на языке C/C++ в среде Linux или Windows.

Общие правила

1. Программа должна корректно работать на реальном оборудовании или в эмуляторе QEMU с пробросом USB-устройства.
2. Для Linux-вариантов требуется доступ к usbfs (root или настроенные udev-правила).
3. Для Windows-вариантов требуется запуск от имени администратора для доступа к низкоуровневым IOCTL.
4. Код должен содержать комментарии и обработку ошибок.

Вариант	Тип устройства	Платформа / API	Требуемая функциональность
1	USB Flash (Mass Storage)	Windows SetupAPI + IOCTL	VID/PID, серийный номер, производитель, объём, буквы дисков, программное извлечение
2	USB HID (мышь/клавиатура)	Linux libusb + sysfs	VID/PID, класс, протокол, количество endpoint'ов, строковые дескрипторы, события подключения
3	USB Mass Storage	Linux libudev + libusb	Полная идентификация, логические тома, тип ФС, программное извлечение с обработкой ошибок
4	Любые USB-устройства	Windows WMI + SetupAPI	Перечисление всех устройств с классификацией по типам, статистика по классам
5	USB Hub + downstream	Linux sysfs	Топология шины (bus.dev.port), адресация, скорость подключения, список downstream-устройств
6	USB-устройства	Кроссплатформа (Win + Lin)	Сравнительная таблица информации, полученной из Windows и Linux для одного и того же устройства
7	Составные устройства	Windows SetupAPI	Разбор всех интерфейсов Composite Device, отдельный вывод для каждого класса внутри одного гаджета
8	Любые USB-устройства	Linux (низкоуровневое)	Мониторинг hotplug в реальном времени через udev, журнал событий с временными метками, сессионная статистика

Вариант 1. USB Flash (Mass Storage) на Windows

Написать приложение, которое:

1. Перечисляет USB-накопители через SetupAPI (GUID_DEVINTERFACE_DISK, GUID_DEVINTERFACE_USB_DEVICE).
2. Для каждого накопителя выводит: VID, PID, серийный номер, производителя, модель, объём.
3. Определяет буквы логических дисков, сопоставленных с накопителем.

Интерфейсы и устройства вычислительных машин

4. Реализует программное безопасное извлечение через CM_Request_Device_Eject.

5. Обрабатывает и выводит причину отказа (veto) при невозможности извлечения.

Вариант 2. USB HID на Linux

Написать приложение, которое:

1. Перечисляет USB HID-устройства (мышь, клавиатура) через libusb.

2. Выводит для каждого устройства: VID, PID, класс, протокол, количество endpoint'ов.

3. Получает и выводит строковые дескрипторы (производитель, продукт, серийный номер).

4. Отслеживает события подключения/отключения через libudev или опрос libusb_get_device_list.

Вариант 3. USB Mass Storage на Linux

Написать приложение, которое:

1. Перечисляет USB Mass Storage-устройства через libudev + libusb.

2. Выполняет полную идентификацию: VID, PID, производитель, продукт, серийный номер, класс, подкласс, протокол.

3. Определяет логические тома (/dev/sdX), тип файловой системы, точки монтирования.

4. Реализует программное безопасное извлечение через udisksctl или authorized=0 в sysfs с обработкой ошибок (открытые файлы, занятые процессы).

Вариант 4. Перечисление и классификация USB-устройств на Windows

Написать приложение, которое:

1. Перечисляет все USB-устройства через SetupAPI и/или WMI (Win32_PnPEntity, Win32_USBHub).

2. Классифицирует устройства по классам (HID, Mass Storage, Audio, Video, Hub и др.).

3. Выводит статистику: количество устройств каждого класса, общее число устройств.

4. Выводит детальную информацию для каждого устройства (FriendlyName, DeviceDesc, Status).

Вариант 5. Топология USB-шины и Hub'ы на Linux

Написать приложение, которое:

1. Анализирует структуру /sys/bus/usb/devices/ и строит дерево шины.

2. Для каждого устройства выводит: адресацию (bus.dev.port), скорость подключения, VID/PID.

3. Для USB Hub'ов выводит список downstream-устройств с указанием портов.

4. Определяет путь от root hub до конечного устройства (цепочка портов).

Вариант 6. Кроссплатформенное сравнение USB-устройства

Написать приложение (или два отдельных скрипта) для Windows и Linux, которое:

1. Для одного и того же физического USB-устройства (выбранного студентом) получает полную идентификационную информацию.

2. Windows: SetupAPI + WMI. Linux: libusb + sysfs.

3. Формирует сравнительную таблицу: VID, PID, серийный номер, производитель, класс, скорость, дескрипторы.

4. Анализирует различия в представлении данных между ОС.

Интерфейсы и устройства вычислительных машин

Вариант 7. Разбор составных устройств (Composite) на Windows

Написать приложение, которое:

1. Находит составные USB-устройства (Composite Device) через SetupAPI.
2. Разбирает все интерфейсы внутри Configuration Descriptor.
3. Для каждого интерфейса выводит: номер интерфейса, класс, подкласс, протокол, количество endpoint'ов.
4. Демонстрирует, что одно физическое устройство содержит несколько логических функций (например, клавиатура + тачпад).

Вариант 8. Мониторинг hotplug USB-устройств на Linux

Написать приложение, которое:

1. Использует libudev для мониторинга событий подключения/отключения USB-устройств в реальном времени.
2. Ведёт журнал событий с временными метками (timestamp): действие (add/remove), VID, PID, производитель.
3. Подсчитывает сессионную статистику: количество подключений, отключений, уникальных устройств за сессию.
4. Выводит текущий список подключённых устройств по запросу (нажатию клавиши).

5. Отладка и диагностика

Linux

```
lsusb                   # Список USB-устройств
lsusb -v                # Подробная информация (дескрипторы)
lsusb -t                # Дерево устройств
udevadm monitor --udev --subsystem-match=usb   # Мониторинг событий


# Информация из sysfs
cat /sys/bus/usb/devices/1-1/idVendor
cat /sys/bus/usb/devices/1-1/idProduct
cat /sys/bus/usb/devices/1-1/manufacturer
cat /sys/bus/usb/devices/1-1/speed


# Информация о дисках
lsblk
blkid
udisksctl status
```

Windows

```
Get-PnpDevice -Class USB
Get-PnpDeviceProperty -InstanceId "USB\VID_xxxx&PID_xxxx\..."
Get-WmiObject Win32_USBHub
Get-WmiObject Win32_DiskDrive | Where-Object {$_.InterfaceType -eq "
USB"}
```

Интерфейсы устройств вычислительных машин

6. Контрольные вопросы

1. Какова топология шины USB и какие роли выполняют корневые и внешние хабы?
2. Опишите иерархию USB-дескрипторов: какие типы существуют и как они связаны между собой?
3. Как получить строковые дескрипторы (производитель, модель, серийный номер) через libusb? Почему для этого необходимо открывать устройство?
4. Чем отличается безопасное извлечение от небезопасного на уровне ОС и файловых систем?
5. Какие механизмы Windows используются для обнаружения подключения USB-устройства в реальном времени? Опишите роль WM_DEVICECHANGE.
6. Что такое составное (*composite*) USB-устройство? Приведите примеры. Как его корректно обработать в приложении?
7. Как определить логические тома, принадлежащие USB-накопителю, в Linux (/sys, lsblk, udev) и Windows (SetupAPI + Volume Management)?
8. Перечислите возможные причины отказа безопасного извлечения устройства и способы их диагностики.
9. Опишите различия между USB 2.0 и USB 3.x на уровне хост-контроллеров (EHCI vs xHCI) и драйверов ОС.
10. Как эмулировать подключение реального USB-устройства в QEMU для тестирования приложения?

Интерфейсы и устройства вычислительных машин